

## DIA 2: SEQUÊNCIAS, ESTRUTURAS CONDICIONAIS, LAÇOS E FUNÇÕES (8h)

### Tópico 01: Coleções Lineares e Condicionais

#### Aula 01: Indexação, Fatiamento e Operadores in / not in

**Descrição Detalhada:** Nesta aula, exploramos a forma como o Python organiza e acessa dados em sequências ordenadas, como listas e strings. A indexação permite acessar elementos individuais utilizando sua posição (índice), onde o primeiro elemento está no índice 0 e índices negativos acessam do final para o início. O fatiamento (slicing) permite extrair subpartes de uma sequência especificando o início, o fim e o passo (sequencia[início:fim:passo]).

Os operadores in e not in são operadores de pertencimento booleanos. O operador in verifica se determinado elemento ou caractere está presente na coleção, retornando True se encontrado e False caso contrário. Por outro lado, not in valida a ausência do elemento, retornando True quando o item não pertence à coleção.

#### Código de Exemplo:

```
# Lista de frutas
frutas = ["maçã", "banana", "uva"]

# Indexação e fatiamento
print("Primeira fruta (índice 0):", frutas[0])
print("Última fruta (índice -1):", frutas[-1])
print("Fatiamento (duas primeiras):", frutas[0:2])

# Uso dos operadores in e not in
print("A fruta 'banana' está na lista?", "banana" in frutas)
print("A fruta 'laranja' NÃO está na lista?", "laranja" not in frutas)

# Exemplo com strings
texto = "Curso de Python"
print("A palavra 'Python' está no texto?", "Python" in texto)
```

**Explicação do Código:** O índice [0] retorna "maçã" e o índice [-1] acessa o último item "uva". A fatia [0:2] pega do índice 0 ao 1. A expressão "banana" in frutas avalia para True porque a string existe na lista, enquanto "laranja" not in frutas também retorna True pois valida que o elemento não faz parte da coleção.

#### Aula 02: Manipulação de Listas e Tuplas

**Descrição Detalhada:** Listas em Python são estruturas dinâmicas e mutáveis, permitindo alteração, adição e remoção de elementos em tempo de execução. Métodos comuns incluem:

- `append(item)`: adiciona um elemento ao final da lista.

- `insert(indice, item)`: insere um elemento em uma posição específica.
- `pop(indice)`: remove e retorna o elemento na posição informada (ou o último, se omitting o índice).
- `remove(valor)`: busca e remove a primeira ocorrência do valor indicado.

Diferente das listas, as **tuplas** são coleções imutáveis, definidas entre parênteses (). Uma vez criadas, não é possível adicionar, remover ou modificar seus elementos. Elas garantem maior segurança e eficiência ao lidar com dados constantes ou registros fixos (como coordenadas, cores RGB ou configurações de sistema).

### Código de Exemplo:

```
# Manipulação de lista mutável
nomes = ["João", "Maria"]
nomes.append("Carlos")          # Adiciona "Carlos" no final
nomes.insert(1, "Ana")          # Insere "Ana" na posição 1
nomes.remove("João")            # Remove o elemento "João"
removido = nomes.pop()          # Remove e retorna o último item
("Carlos")

print("Lista final de nomes:", nomes)
print("Item removido pelo pop:", removido)

# Declaração de tupla imutável
cores_primarias = ("vermelho", "verde", "azul")
print("Cores (Tupla):", cores_primarias)

# Tentar alterar uma tupla causaria um erro (TypeError)
# cores_primarias[0] = "amarelo" # Inválido!
```

**Explicação do Código:** A lista `nomes` é alterada dinamicamente por meio dos métodos `append`, `insert`, `remove` e `pop`. Já a tupla `cores_primarias` é criada como um agrupamento imutável e seguro de valores fixos.

## Aula 03: Tomada de Decisão (if, elif, else e match/case)

**Descrição Detalhada:** As estruturas condicionais controlam o fluxo de execução do programa com base em condições booleanas. O bloco básico utiliza `if` para testar uma condição primária, `elif` para verificar condições alternativas caso as anteriores sejam falsas, e `else` como um bloco de captura final quando nenhuma condição anterior for atendida.

A partir do Python 3.10, foi introduzido o operador `match/case` (Structural Pattern Matching), ideal para criar seleções estruturadas e menus de opções de forma legível e elegante. O caractere sublinhado `case _:` atua como o padrão curinga (wildcard), equivalente ao bloco `else` tradicional.

### Código de Exemplo:

```
# Estrutura condicional clássica (if/elif/else)
idade = 18
```

```

if idade < 12:
    print("Categoria: Criança")
elif idade < 18:
    print("Categoria: Adolescente")
else:
    print("Categoria: Adulto")

# Menu estruturado usando match/case
opcao = int(input("Escolha uma opção (1-Cadastrar, 2-Buscar, 3-Sair): "))

match opcao:
    case 1:
        print(">> Opção 'Cadastrar' selecionada.")
    case 2:
        print(">> Opção 'Buscar' selecionada.")
    case 3:
        print(">> Saindo do programa...")
    case _:
        print(">> Opção inválida! Tente novamente.")

```

**Explicação do Código:** A primeira parte classifica a faixa etária comparando o valor da variável idade. Na segunda parte, a instrução match avalia a escolha numérica do usuário no menu, executando o bloco do case correspondente ou caindo no caso padrão case \_: se nenhuma opção válida for digitada.

## Tópico 02: Laços de Repetição e Funções

### Aula 04: Laços for (range) e while

**Descrição Detalhada:** Estruturas de repetição permitem executar um bloco de código múltiplas vezes:

- **Laço for:** Utilizado principalmente para iterações contadas ou para percorrer sequências. A função integrada range(início, fim, passo) gera uma sequência numérica onde o valor final é exclusivo (não incluído).
- **Laço while:** Utilizado quando o número de repetições não é previamente conhecido, repetindo o bloco de instruções enquanto uma determinada condição permanecer como verdadeira (True). É fundamental atualizar as variáveis de controle dentro do laço para evitar loops infinitos.

#### Código de Exemplo:

```

# Laço for percorrendo range com parâmetro de passo
print("=== Exemplo com for ===")
for i in range(1, 6, 2):
    print(f"Número ímpar: {i}")

```

```
# Laço while condicionado
print("\n=== Exemplo com while ===")
contador = 3
while contador > 0:
    print(f"Contagem regressiva: {contador}")
    contador -= 1 # Decrementa o contador

print("Fim da contagem!")
```

**Explicação do Código:** O laço for utiliza range(1, 6, 2) para gerar os números 1, 3 e 5. O laço while executa enquanto contador > 0, decrementando o valor a cada iteração até que a condição se torne falsa.

## Aula 05: Funções com Parâmetros e Retorno (def)

**Descrição Detalhada:** Funções são blocos de código reutilizáveis definidos com a palavra-chave def. Elas ajudam a modularizar o sistema, evitando repetição de código e facilitando a manutenção.

As funções podem receber entradas chamadas **parâmetros** e processá-las. A instrução return envia o valor resultante do processamento de volta ao chamador. Caso não haja uma instrução return explícita, a função retornará por padrão o valor None.

### Código de Exemplo:

```
# Definição da função com parâmetros
def calcular_media(nota1, nota2):
    media = (nota1 + nota2) / 2
    return media

def verificar_situacao(media):
    if media >= 7.0:
        return "Aprovado"
    else:
        return "Recuperação"

# Chamada das funções e armazenamento dos retornos
res_media = calcular_media(8.5, 6.5)
situacao = verificar_situacao(res_media)

print(f"Média calculada: {res_media:.2f}")
print(f"Situação do aluno: {situacao}")
```

**Explicação do Código:** A função calcular\_media recebe duas notas, realiza o cálculo da média aritmética e devolve o valor numérico com return. Esse valor é repassado à função verificar\_situacao, que valida se o aluno está aprovado ou em recuperação.

## Lista de Exercícios Práticos

### Atividades Específicas por Aula

1. **Atividade 01 (Aula 01 - Indexação e Pertencimento):** Crie uma lista chamada `linguagens = ["Python", "Java", "C++", "JavaScript", "Go"]`. Faça um programa que:
  - Exiba o terceiro elemento e o último elemento utilizando índice negativo.
  - Verifique e exiba se a linguagem "Python" está presente na lista.
  - Verifique e exiba se a linguagem "Ruby" *não* está presente na lista.
2. **Atividade 02 (Aula 02 - Manipulação de Listas e Tuplas):** Crie uma lista vazia chamada `carrinho`.
  - Adicione três produtos usando `append()`.
  - Insira um novo produto na primeira posição usando `insert()`.
  - Remova um produto pelo seu nome usando `remove()` e outro pelo seu índice usando `pop()`.
  - Defina uma tupla chamada `impostos_fixos = (0.05, 0.12, 0.18)` e exiba os itens na tela.
3. **Atividade 03 (Aula 03 - Estruturas Condicionais e Match/Case):** Escreva um algoritmo de caixa eletrônico que solicite um código de operação (1 - Verificar Saldo, 2 - Depositar, 3 - Sacar, 4 - Sair) e utilize a estrutura `match/case` para exibir uma mensagem personalizada para cada ação escolhida ou de opção inválida.
4. **Atividade 04 (Aula 04 - Laços de Repetição):**
  - Utilizando o laço `for` e `range`, imprima a tabuada de multiplicação do número 7 (de 1 a 10).
  - Utilizando um laço `while`, peça para o usuário digitar uma senha até que ele acerte o valor pré-definido "python123".
5. **Atividade 05 (Aula 05 - Funções com Retorno):** Crie uma função chamada `converter_temperatura(celsius)` que receba a temperatura em graus Celsius e retorne o valor convertido para Fahrenheit usando a fórmula:  $F = (C * 9/5) + 32$ . Em seguida, chame a função passando um valor e imprima o resultado.

### Desafios Integradores (Englobando todo o conteúdo)

1. **Desafio 01 - Sistema de Gerenciamento de Estoque Modular:** Crie um programa com um menu principal (usando `while` e `match/case`) que permita gerenciar uma lista de produtos chamada `estoque`.
  - Crie uma função `adicionar_produto(estoque, produto)` que adiciona o item apenas se ele ainda não estiver presente na lista (use o operador `in`).
  - Crie uma função `remover_produto(estoque, produto)` que remove o item se ele existir na lista.
  - Crie uma função `listar_produtos(estoque)` que utilize um laço `for` para exibir todos os itens numerados.
  - O sistema deve repetir o menu até que o usuário escolha a opção de sair.
2. **Desafio 02 - Análise e Processamento de Notas de Turma:** Crie um programa

que cadastre uma sequência de alunos e suas respectivas notas finais.

- Armazene cada registro como uma tupla no formato (nome, nota) dentro de uma lista de alunos.
- Utilize um laço while para permitir o cadastro contínuo até que o usuário digite "fim" no nome do aluno.
- Crie uma função analisar\_turma(lista\_alunos) que percorra a lista com um laço for e:
  - Calcule a média geral da turma.
  - Identifique a maior e a menor nota.
  - Retorne uma lista contendo apenas os nomes dos alunos aprovados (nota  $\geq 7.0$ ).
- Exiba o relatório final completo com todas essas estatísticas organizadas.